MODULE 11 · DOMAIN 1 - MONITORING, LOGGING, ANALYSIS, REMEDIATION, AND PERFORMANCE OPTIMIZATION

Observability Beyond CloudWatch

X-Ray, OpenTelemetry, Application Signals, Managed Prometheus, Managed Grafana and the Health Dashboard.

DOMAIN 1 of 5 EXAM WEIGHT 22% MODULE 11 of 35

admin@sthithapragna.com

Three pillars, one question each. **Metrics**: is it healthy? **Logs**: what happened? **Traces**: where did the time go?

AWS X-Ray

- Follows one request across services using a **trace ID**, made of **segments** from each service and **subsegments** within them.
- The **service map** draws the call graph automatically, with latency and error rates on each edge.
- **Annotations are indexed** and can be filtered on. **Metadata is not indexed**, so it is for context you read, not search.
- **Sampling rules** control cost: by default one request per second plus a percentage of the rest.

> ON THE EXAM
>
> Annotation versus metadata is a favourite detail. If a question needs to filter traces by customer ID, that value must be an annotation.

Currency: the X-Ray SDKs are in maintenance mode

- **The X-Ray SDKs and the X-Ray daemon formally entered maintenance mode in 2026.** Security fixes only, no new features.
- **The X-Ray service itself is fully supported.** Console, trace processing and backend capabilities are unchanged.
- The instrumentation path forward is **OpenTelemetry, via the AWS Distro for OpenTelemetry**.
- Migration guides exist for Java, Python, Node.js, .NET, Go and Ruby, with both zero-code and manual instrumentation.

> ON THE EXAM
>
> Be precise about the boundary. The SDKs are frozen; the service is not. Do not tell an interviewer that X-Ray is deprecated.

AWS Distro for OpenTelemetry

- **ADOT** is AWS's supported build of the OpenTelemetry Collector. One agent collects **metrics, traces and logs**.
- Runs on **EKS, ECS, Lambda and EC2**, as a sidecar, a DaemonSet or a Lambda layer.
- Sends to **CloudWatch, X-Ray, Managed Service for Prometheus and OpenSearch**. Changing destination is a config change.
- **Vendor-neutral instrumentation.** Instrument once, and the backend becomes a decision you can revisit.

> ON THE EXAM
>
> "Avoid vendor lock-in" or "one agent for metrics and traces" points at ADOT. That is the whole design goal.

CloudWatch Application Signals

- **Auto-instruments** applications on EKS, ECS, EC2 and Lambda with no code change, using OpenTelemetry underneath.
- Produces standard **service-level metrics**: latency, faults, errors and call volume, plus a service map.
- Define **Service Level Objectives** against those metrics, and it tracks **error budget** burn for you.
- 2026 additions: **SLO recommendations** from historical performance, **service-level SLOs**, and an **SLO performance report**.

> ON THE EXAM
>
> X-Ray shows you one request. Application Signals shows you the service's health as a whole, and whether it is meeting its target.

X-Ray or Application Signals?

X-RAY WHEN

One request, in detail

- Where did the time go in this trace?
- Segment and subsegment level detail
- Filter by annotation to find specific traces
- Debugging a specific slow path

APPLICATION SIGNALS WHEN

One service, over time

- Is this service meeting its objective?
- Standard golden metrics, no code change
- SLOs and error budget tracking
- Ongoing operations rather than debugging

THE TIEBREAKER

They are layers, not alternatives. Application Signals gives you the health view; X-Ray gives you the trace behind a bad one.

Amazon Managed Service for Prometheus

- A managed, **Prometheus-compatible** metrics store. You create a **workspace** and write to it.
- Query with **PromQL**, the language teams already know from open-source Prometheus.
- Ingest via **remote write** from your own Prometheus, from ADOT, or with the **managed collector** that scrapes EKS for you.
- Includes **recording rules and alerting rules**, and an Alert Manager, so alerting need not go through CloudWatch.

> ON THE EXAM
>
> "We already use Prometheus and PromQL and do not want to rewrite our dashboards and alerts" is exactly this service.

Amazon Managed Grafana

- Managed **open-source Grafana**. Workspaces, dashboards, alerting, and authentication through **IAM Identity Center** or SAML.
- Its strength is **many data sources on one dashboard**: CloudWatch, Managed Prometheus, OpenSearch, X-Ray, Timestream, Athena, and third parties.
- Ideal for a **single pane across AWS and non-AWS** systems, including on-premises and other clouds.
- 2026: **Grafana 12.4** workspaces, customer managed KMS keys, and GovCloud availability.

> ON THE EXAM
>
> The decisive word is heterogeneous. CloudWatch dashboards graph AWS well; Grafana graphs AWS and everything else together.

When logs outgrow CloudWatch Logs

- **Amazon OpenSearch Service** is the answer for full-text search, long retention and complex log analytics at scale.
- Get logs there with a **subscription filter**, directly or through Amazon Data Firehose. Module four covered the mechanism.
- Its **Observability plugin** adds trace analytics, and it can now query Managed Prometheus metrics with native PromQL.
- It is a **cluster you size and pay for**. CloudWatch Logs is serverless, so do not reach for OpenSearch by default.

> ON THE EXAM
>
> Athena over exported logs in S3 is the cheaper alternative when queries are occasional rather than interactive.

CloudWatch dashboards or Managed Grafana?

CLOUDWATCH DASHBOARDS

AWS-native and simple

- No extra service to run or pay for
- Cross-account through the sink and link model
- Alarms and Logs Insights widgets built in
- Limited to CloudWatch data

MANAGED GRAFANA

Heterogeneous and rich

- Many data sources, AWS and not, on one panel
- Existing Grafana dashboards work unchanged
- Richer visualisations and templating
- Charged per active user per month

THE TIEBREAKER

Only AWS data and a small team: CloudWatch. Existing Grafana dashboards, or data from outside AWS: Managed Grafana.

Synthetics or RUM?

CLOUDWATCH SYNTHETICS

Robots, all the time

- Canaries run scripted journeys on a schedule
- Detects outages with no users present
- Blueprints: heartbeat, API, broken links, visual diff
- Screenshots and HAR files on failure

CLOUDWATCH RUM

Real users, real browsers

- A JavaScript snippet in the page
- Actual page load times, errors, device and geography
- Session replay for reproducing a reported issue
- Only shows what real traffic exercised

THE TIEBREAKER

Synthetics catches an outage at 3am on a page nobody visited. RUM tells you the checkout is slow on Android in Australia.

Synthetics canaries in practice

- A **canary** is a Node.js or Python script on a schedule. **Blueprints** cover heartbeat, API, broken links, visual diff and a recorder.
- Each run stores **screenshots, HAR files and logs** in S3, so a failure at 3am is still reproducible in the morning.
- Emits **SuccessPercent** and **Duration** metrics, so an alarm on a canary is an ordinary CloudWatch alarm.
- Canaries can run **inside a VPC**, which is how you monitor an internal application with no public endpoint.

> ON THE EXAM
>
> The visual monitoring blueprint compares screenshots against a baseline, catching a broken layout that returns HTTP 200 perfectly happily.

AWS Health Dashboard

- **Service health** is the public view of AWS service status. **Your account health** is personalised to your resources.
- Publishes events to the **default EventBridge bus**, so scheduled changes can trigger automation. Module eight used this.
- **Organizational view** aggregates health events across every account in the organisation.
- The **Health API** requires a **Business, Enterprise On-Ramp or Enterprise** Support plan. The console view does not.

> ON THE EXAM
>
> Added to the in-scope list in exam guide v1.1. It is the only service that warns you before anything has actually broken.

Where AWS is heading, in one slide

- **OpenTelemetry is the instrumentation standard.** X-Ray SDKs frozen, ADOT everywhere, OTLP ingestion into CloudWatch in preview.
- **PromQL is becoming a first-class CloudWatch language**, alongside Logs Insights QL and Metrics Insights.
- **Container Insights is moving to OpenTelemetry** collection for EKS.
- **CloudWatch Evidently reached end of life in October 2025.** If older material mentions it, that material is stale.

> ON THE EXAM
>
> You do not need preview features for the exam. You do need to recognise the direction, because it explains why answers now favour ADOT.

Which tool answers which question

THE QUESTION	THE TOOL
Is this service meeting its target?	Application Signals SLOs
Where did the time go in this request?	X-Ray
What did real customers experience?	CloudWatch RUM
Is the site up when nobody is looking?	CloudWatch Synthetics
We already have PromQL rules and dashboards	Managed Prometheus and Managed Grafana
One dashboard across AWS and elsewhere	Amazon Managed Grafana
Is AWS about to change something under us?	AWS Health Dashboard

ON THE EXAM

Read a scenario for the question being asked rather than the technology mentioned. The question maps to exactly one row far more often than not.

A team is instrumenting a new set of microservices on EKS in 2026. They want distributed tracing into X-Ray and metrics into CloudWatch, and they may later move metrics to a third-party backend without re-instrumenting.

An engineer proposes adding the AWS X-Ray SDK to each service, plus a separate metrics library.

What should the team do instead?

A Proceed with the X-Ray SDK, since X-Ray is the AWS-native tracing service

B Instrument with OpenTelemetry and deploy the ADOT Collector, exporting traces to X-Ray and metrics to CloudWatch

C Use the CloudWatch agent DaemonSet only, and rely on Container Insights for tracing

D Deploy Amazon Managed Service for Prometheus and use PromQL for traces

◆ PAUSE AND DECIDE

What should the team do instead?

A X-Ray SDK

The SDKs are in maintenance mode: security fixes only. Building new instrumentation on a frozen library, and it locks the backend in.

B OpenTelemetry with ADOT

One agent for traces and metrics, X-Ray and CloudWatch as exporters today, and a different backend later is a config change.

C CloudWatch agent only

Container Insights gives cluster and container metrics. It does not produce distributed traces, which is half the requirement.

D Prometheus for traces

Prometheus is a metrics system and PromQL is a metrics language. Neither handles traces.

THE SIGNAL

Two clauses: tracing into X-Ray, and a possible backend change later. ADOT satisfies both, and A satisfies neither well.

> A company runs workloads on AWS, in an on-premises data centre, and in another cloud. Three teams each maintain their own dashboards. Leadership wants one dashboard showing service health across all three environments, and the on-premises team already has extensive Grafana dashboards backed by Prometheus.
>
> The company wants to reuse existing work where possible.

What should the company implement?

A. Recreate every dashboard in CloudWatch and use cross-account observability

B. Amazon Managed Grafana, with CloudWatch, Amazon Managed Service for Prometheus and the existing Prometheus as data sources

C. Export all metrics to S3 and build dashboards in Amazon QuickSight

D. Install the CloudWatch agent on every on-premises and other-cloud server, and use CloudWatch dashboards

◆ PAUSE AND DECIDE

What should the company implement?

A Recreate in CloudWatch

Throws away the existing Grafana work the scenario asks to reuse, and cross-account observability is within AWS only.

B Managed Grafana with several data sources

One dashboard across all three environments, existing Grafana dashboards work unchanged, and the existing Prometheus stays a data source.

C S3 plus QuickSight

QuickSight is business intelligence over datasets, not operational monitoring. No alerting, no live metrics.

D CloudWatch agent everywhere

Technically possible, and it means re-plumbing every non-AWS system and abandoning the Grafana dashboards. Maximum effort, least reuse.

THE SIGNAL

"Heterogeneous" plus "reuse existing work" is the Managed Grafana signature. Both halves point the same way.

> An e-commerce checkout page is reported as slow by a small number of customers, mostly in one country. CloudWatch shows the load balancer and application servers well within normal latency, and Application Signals reports the checkout service is meeting its SLO.
>
> The team needs to understand what those customers actually experienced, including whether the page rendered correctly.

What should the team implement?

A. CloudWatch Synthetics canaries running the checkout journey every minute

B. CloudWatch RUM, with session replay enabled

C. X-Ray with a sampling rule capturing 100 percent of checkout requests

D. Enable detailed monitoring on the application servers

◆ PAUSE AND DECIDE

What should the team implement?

A Synthetics canaries

Canaries run from AWS on a schedule. They cannot reproduce a real customer's browser, device or network in a specific country.

B RUM with session replay

Reports what real browsers experienced, broken down by geography and device, and replay shows whether the page actually rendered.

C 100 percent X-Ray sampling

Traces stop at the server. Every server-side measurement is already normal, so more of the same data will not explain it.

D Detailed monitoring

One-minute instead of five-minute server metrics. The server metrics are already fine; frequency does not change that.

THE SIGNAL

Server-side is healthy and users are not. That gap is the definition of what RUM measures and nothing server-side can see.

A platform team runs 40 EKS clusters and has years of Prometheus recording rules, alerting rules and Grafana dashboards written in PromQL. They want to stop running and scaling their own Prometheus servers, without rewriting any of that work.

They also want a managed way to scrape the clusters rather than running their own collectors.

What should the team adopt?

A Amazon Managed Service for Prometheus with its managed collector, queried from Amazon Managed Grafana

B Convert all PromQL rules to CloudWatch metric math and Container Insights

C CloudWatch Application Signals on every cluster

D Self-managed Prometheus on EKS, with remote write to CloudWatch

◆ PAUSE AND DECIDE

What should the team adopt?

A Managed Prometheus plus managed collector

PromQL rules and Grafana dashboards carry over unchanged, the managed collector scrapes the clusters, and nothing is self-hosted.

B Convert PromQL to metric math

Exactly the rewrite the scenario forbids. Years of rules, translated by hand, with every translation a chance to introduce a bug.

C Application Signals

Excellent for standard service metrics with no code change, and it does not run PromQL or accept existing Prometheus rules.

D Self-managed Prometheus

Keeps the operational burden the scenario is trying to remove. They still run and scale Prometheus themselves.

THE SIGNAL

"Without rewriting" is the constraint. Only a PromQL-compatible store preserves recording rules, alerting rules and dashboards intact.

> An internal HR application runs on private subnets with no public endpoint. It was unavailable for six hours over a weekend and nobody noticed until Monday morning. Server metrics stayed normal throughout, because the servers were healthy; a dependency the login page called had failed.
>
> The team wants to detect this class of failure automatically, in a way that also proves the login journey works end to end.

What should the team implement?

A. A CloudWatch Synthetics canary running the login journey on a schedule, configured to run inside the VPC

B. CloudWatch RUM on the login page

C. A composite alarm across all server-level metrics for the application

D. An AWS Health Dashboard EventBridge rule for the application's Region

◆ PAUSE AND DECIDE

What should the team implement?

A VPC canary running the login journey

Exercises the whole journey on a schedule whether or not anyone is using it, and canaries can run inside the VPC to reach a private endpoint.

B RUM on the login page

RUM only reports what real users generate. At 2am on a Saturday there are no users, so it would have stayed silent exactly as everything else did.

C Composite alarm on server metrics

The scenario states server metrics were normal throughout. Combining metrics that were all healthy produces an alarm that stays in OK.

D Health Dashboard rule

Health reports AWS-side events. A failed application dependency is your problem, and AWS has nothing to report about it.

THE SIGNAL

"Nobody noticed until Monday" is the Synthetics signature. RUM needs users; a weekend outage is precisely when there are none.

The five things to carry forward

- **X-Ray SDKs and daemon are in maintenance mode; the X-Ray service is not.** New instrumentation uses ADOT.
- **Application Signals for service health and SLOs; X-Ray for one request.** Layers, not alternatives.
- **Existing PromQL and Grafana work means Managed Prometheus and Managed Grafana.** Rewriting is the wrong answer.
- **Synthetics is robots on a schedule; RUM is real browsers.** Healthy server plus unhappy users means RUM.
- **AWS Health warns before anything breaks**, and publishes to EventBridge.

> ON THE EXAM
>
> That completes domain one. Next: Auto Scaling in depth, and the start of domain two.